Muthukumaran L

Scale_Invariant_Sharpness_Metrics_and_Their_Relationship...

BMS Institute of Technology, Bengaluru

Document Details

Submission ID
trn:oid:::3618:137191552

Submission Date
Apr 30, 2026, 2:28 PM GMT+5:30

Download Date
Apr 30, 2026, 2:31 PM GMT+5:30

File Name
Scale_Invariant_Sharpness_Metrics_and_Their_Relationship_with_Generalization_in_Deep_Neural ....pdf

File Size
837.8 KB

8 Pages

4,969 Words

25,782 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Scale-Invariant Sharpness Measures and Their Impact on Generalization in Deep Neural Networks

Author One
Dept. of Computer Science and Engineering
BMS Institute of Technology & Management
author1@bmsit.in

Author Two
Dept. of Computer Science and Engineering
BMS Institute of Technology & Management
author2@bmsit.in

Author Three
Dept. of Computer Science and Engineering
BMS Institute of Technology & Management
author3@bmsit.in

***Abstract*—Curvature measures of sharpness based on Hessian eigenvalues can be affected by a ReLU symmetry, where scaling adjacent layers does not change the network's output but $\lambda_{\max}(H)$ is multiplied by an arbitrary factor. Consequently, such curvature values make comparison across optimization algorithms and architectures with iterates settling at distinct scales difficult. In contrast to that, $S_{\text{norm}}(\theta)=\lambda_{\max}(H)/\|\theta\|^2$, which accounts for the uniform part of the symmetry, is used in our study. The following test compares the quality of prediction for curvature vs. S_{norm} as a metric of sharpness. Both the MLP on MNIST and the ResNet-18 on CIFAR-10 are trained using three optimizers: SGD, Momentum-SGD, and Adam, using five random seeds for each. The situation on MNIST is straightforward. S_{norm} correlates well with test accuracy, Pearson $r=-0.961$, outperforming even raw $\lambda_{\max}$ in terms of $r=-0.943$. Adam optimizer finds the flatest point with $\lambda_{\max}=2.84$ and achieves 97.88% of accuracy. The story on CIFAR-10 flips. Momentum-SGD operates in basins around $5.6\times$ sharper than those found by SGD, however, the former algorithm still outperforms SGD by around ten percent of accuracy; all curvature–accuracy correlations are below 0.31 in absolute values, and the sign changes. We understand this finding as evidence that the relation between curvature and generalization is contingent rather than intrinsic. Four specific conclusions are listed below, together with the idea of extending S_{norm} to layer-wise.**

***Index Terms*—flat minima, sharpness, Hessian curvature, scale-invariant metrics, generalization, optimizer dynamics, loss landscape, deep learning.**

I. Introduction

Deep nets generalize to unseen data with humiliating ease, despite being trained in regimes whose complexity theory predicts catastrophe [**?**], [**?**]. The reason behind this is not fully understood, but the oldest geometric intuition is the notion of the *flat minima hypothesis* [**?**]: optima located in wider, flatter regions of the loss function can be encoded more compactly and are expected, for that reason alone, to generalize better.

This connection gained renewed interest when Keskar et al. [**?**] made the relation between batch size and sharpness explicit, noting that large batches tended to settle into sharper minima and failed to maintain generalization in practice. Foret et al. [**?**] made the next leap and directly trained on flatness through the Sharpness-Aware Minimization (SAM) technique, in which each iterate is first perturbed toward its neighbor of worst function value. Two well-aimed critiques sit alongside this line of work, and have not been answered together.

The first is algebraic, due to Dinh et al. [**?**]. ReLU networks admit a layer-wise symmetry: scale layer ℓ's weights by $\alpha > 0$, divide layer $\ell{+}1$'s by α, and the function is unchanged—but the Hessian eigenvalues at that point shift by α^{-2}. A reparameterization, in other words, can make any solution look arbitrarily sharp or flat without changing its behavior. The second critique is empirical, from Jiang et al. [**?**]: across a large sweep of architectures and tasks, curvature-based measures rank models inconsistently and often no better than chance.

What is missing: Both critiques have been stated; neither has been put to a controlled test alongside a concrete, scale-aware fix. Notwithstanding, no previous work attempts (a) to analyze a metric for curvature that is resistant to rescaling, end-to-end, and (b) to understand the context under which curvature does or does not correlate with generalization, using matching methods. Otherwise, a user will not be able to determine whether a discrepancy in sharpness between two runs indicates a geometric phenomenon or just an issue with parameter bookkeeping.

Contributions: This paper offers four.

C1. We study $S_{\text{norm}}(\theta)=\lambda_{\max}(H)/\|\theta\|^2$, a normalized sharpness measure that is invariant under uniform rescaling of θ and partially corrects the broader layer-wise ReLU symmetry (Section III). S_{norm} is related in spirit to the adaptive sharpness used inside ASAM [**?**], but is meant for post-hoc *measurement* rather than training, and uses a single global norm.

C2. A controlled experiment—MLP on MNIST, ResNet-18 on CIFAR-10, three optimizers, five seeds—records $\lambda_{\max}$, $\text{Tr}(H)$, and S_{norm} at convergence under tightly matched training protocols (Section V).

C3. We document a clear dataset-complexity boundary (Section VI). On MNIST every metric correlates strongly and negatively with test error, with S_{norm} at $r=-0.961$. On CIFAR-10 the picture collapses: every correlation shrinks below 0.31 and flips sign. Scale normalization improves MNIST but does *not* rescue CIFAR-10, suggesting the failure there is structural rather than a scale artifact.

C4. We give four practitioner recommendations (Section VIII) grounded in the breakdown patterns, and identify a layer-wise version of S_{norm} as the most useful next step.

On the scope of our work. We are not suggesting a novel optimization algorithm, nor do we make the claim that S_{norm} predicts generalization in general. The empirical scope of our study is deliberately limited—two datasets, two architectures,

no augmentation—in order to isolate the role of curvature. Extensions are explored in Section IX.

II. Related Work

A. The Flat-Minima Idea and Its Empirical Anchors

The flat-minima theory is rooted in a paper by Hochreiter and Schmidhuber [**?**]. In the context of minimum description length,they argued that since low-curvature optima contain less information, they are more likely to generalize. Back then, the discussion was largely theoretical. Almost twenty years later, Keskar et al. [**?**] provided a practical way to explore the issue, using batch size as a knob for controlling how sharp the optimum obtained by an optimizer is. As it turned out, sharp optima resulted in poorer generalization.

B. The Scale-Invariance Critique

In a groundbreaking work, Dinh et al. [**?**] discovered a fundamental issue in the structure of all eigenvalue-based sharpness estimators for ReLU networks. Owing to positive homogeneity, there exist an infinite number of weight vectors representing the same function, where the Hessian eigenvalues at those points are scaled differently by the square of the ratio. Therefore, reparameterization is capable of arbitrarily tuning the sharpness without altering predictions made by the model. Moreover, in [**?**], Jiang et al. extended the critique to generalization measures, discovering that the overwhelming majority, including sharpness estimators, were poor ranking methods.

C. The Eigenvalue Spectrum at Convergence

Sagun et al. [**?**] found that the Hessian spectrum following training exhibits a dual layer: a few large eigenvalues, associated with the task signal, and a bulk of near-zero values. This is precisely why we can consider $\lambda_{\max}$ as a valid proxy for the effective curvature, because the contribution of the bulk is negligible. Fort and Ganguli [**?**], on the other hand, tracked the evolution of the Hessian spectrum during training, revealing distinct timescales between the two sets of eigenvalues.

D. Optimizing for Flatness

SAM [**?**] proposed by Foret et al. shifted the problem of diagnosing sharpness to that of minimizing it. This method applies a perturbation to the current iterate towards the locally most deteriorated neighboring point before applying the gradient step. It involves an extra pass per step, but guarantees performance improvements on common benchmarks. ASAM [**?**] proposed by Kwon et al. attempts to solve the rescaling problem raised by Dinh within the optimizer by scaling the perturbation by the magnitude of each parameter separately. ASAM is the most similar previous work compared to our own. The main difference is that ASAM normalizes the *perturbation* in training, whereas S_{norm} normalizes the *measurement* after training.

E. Trajectory Effects, Stability, and Other Geometry

The generalization error of SGD was upper bounded by Hardt et al. [**?**] through the concept of algorithmic stability using a bound that does not take the curvature of the loss surface into account but relies instead on the number of training steps and the step-size used during learning. The message conveyed is that not only the destination in parameter space affects the behavior on test data but also the journey there. The method called Entropy-SGD [**?**], developed by Chaudhari et al., enshrines this bias into its update rule. The work by Li et al. [**?**] provided two-dimensional representations of the loss landscape that made it easy to grasp the distinction between valleys and peaks. Belkin et al. [**?**] have thrown a wrench into the works with their double descent phenomenon.

III. Theoretical Background

A. Local Geometry at a Converged Solution

Let $f_\theta : \mathcal{X} \to \mathcal{Y}$ be a d-parameter network, $\mathcal{D}=\{(x_i, y_i)\}_{i=1}^N$ the training set, and ℓ the per-example cross-entropy loss. The training objective is

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\theta(x_i), y_i\big). \tag{1}$$

At a stationary point θ^* the gradient vanishes, so expanding L to second order around θ^* gives

$$L(\theta^*+\epsilon) \approx L(\theta^*) + \tfrac{1}{2}\, \epsilon^\top H(\theta^*)\, \epsilon, \tag{2}$$

with $H(\theta^*) = \nabla^2_\theta L(\theta^*)$. The local curvature H controls how steeply the training loss climbs as the parameters are pushed away from θ^*. Following [**?**], we take sharpness to be the worst-case rise inside a ball of radius ρ:

$$S_\rho(\theta^*) = \max_{\|\epsilon\|_2 \le \rho} \big[L(\theta^*+\epsilon) - L(\theta^*)\big]. \tag{3}$$

By (2), S_ρ scales with the largest eigenvalue of H, which is what makes $\lambda_{\max}(H)$ a natural—and computable—proxy. A PAC-Bayes bound from [**?**] then connects curvature and parameter scale to the test error:

$$\mathbb{E}_{\text{test}} \;\le\; \mathbb{E}_{\text{train}} + \mathcal{O}\left(\sqrt{\frac{\|\theta\|^2}{n\rho^2}}\right). \tag{4}$$

Smaller sharpness and smaller parameter norm both contribute to tightening the bound.

B. Why Raw Eigenvalues Are Unreliable

For ReLU networks, multiplying layer ℓ's weights by $\alpha > 0$ and dividing layer $\ell{+}1$'s by α leaves the function intact. The Hessian, however, transforms:

$$f_\theta(x) = f_{\alpha\cdot\theta}(x), \qquad H_{\alpha\cdot\theta} = \alpha^{-2}\, H_\theta. \tag{5}$$

The α^{-2} factor is the catch. Choosing α inflates or deflates $\lambda_{\max}$ at will, with zero functional consequence. Any comparison of $\lambda_{\max}$ across configurations whose parameter scales differ is therefore ambiguous on its face.

TABLE I
COMPARATIVE SURVEY OF RELATED WORK ON SHARPNESS AND GENERALIZATION.

#	Paper (venue)	Focus	Metric	Key finding	Limitation	Gap our work fills
1	Hochreiter & Schmidhuber [?] (1997)	MDL flatness theory	Curvature	Flat basins linked to robustness	No large-scale validation	Modern empirical evaluation
2	Keskar et al. [?] (ICLR 2017)	Batch size vs. sharpness	Sharpness, accuracy	Large batch ⇒ sharp minima	Metric is scale-sensitive	Scale-invariant replacement
3	Dinh et al. [?] (ICML 2017)	Rescaling critique	Hessian sharpness	Sharp minima can generalize well	No replacement proposed	S_{norm} as a concrete fix
4	Li et al. [?] (NeurIPS 2018)	Landscape visualization	2D projections	Complex geometric structure	2D projections only	Quantitative metrics
5	Foret et al. [?] (ICLR 2021)	SAM optimization	Sharpness, accuracy	Flatness as objective helps	Extra compute	Diagnostic counterpart
6	Kwon et al. [?] (ICML 2021)	ASAM (adaptive SAM)	Adaptive sharpness	Per-parameter scaling helps SAM	Tied to optimizer	Post-hoc measurement variant
7	Sagun et al. [?] (2016)	Hessian spectrum	Eigenvalues	Two-tier spectrum at convergence	Single optimizer	Multi-optimizer study
8	Hardt et al. [?] (ICML 2016)	Stability bounds	Stability	Trajectory bounds generalization	Not curvature-focused	Trajectory vs. endpoint
9	Chaudhari et al. [?] (ICLR 2017)	Entropy-SGD	Flatness	Wide-valley bias is achievable	Limited adoption	Optimizer comparison
10	Jiang et al. [?] (ICLR 2020)	Generalization measures	Multiple	Most measures weak predictors	No Hessian focus	Curvature-only evaluation
This work	—	Controlled study	Three metrics	Dataset-conditional link; S_{norm} improves prediction	Two datasets	S_{norm} + complexity boundary

C. A Scale-Normalized Curvature Measure

The bound in (4) already pairs sharpness with $\|\theta\|^2$. That structure suggests forming the ratio directly, which has the side effect of canceling the α^{-2} factor in (5):

$$S_{\text{norm}}(\theta) = \frac{\lambda_{\max}(H)}{\|\theta\|^2}, \qquad S_{\text{norm}}(\alpha\theta) = S_{\text{norm}}(\theta) \;\; \forall\, \alpha > 0. \tag{6}$$

S_{norm} is invariant under uniform rescaling of θ, which addresses the most common version of the rescaling artifact: two optimizers that converge at different parameter magnitudes can be compared directly.

One important caveat: The invariance in (6) holds for a single uniform α. The full ReLU symmetry of (5) permits independent layer-wise factors α_ℓ that cancel across adjacent layers, and a global division by $\|\theta\|^2$ corrects only the uniform projection of that family. We therefore present S_{norm} as a useful but partial fix; a layer-wise refinement is an open direction (Section IX). This caveat matters because the empirical advantage of S_{norm} over raw $\lambda_{\max}$ in our MNIST experiments turns out to be modest—which is consistent with the fact that the global division removes only one component of the underlying scale ambiguity.

IV. METHODOLOGY

A. What We Measure at Convergence

Three quantities are computed once training is finished, on a fixed held-out batch of 512 examples. The batch is set aside before training begins and reused identically across every configuration.

Dominant eigenvalue $\lambda_{\max}(H)$:

$$\lambda_{\max}(H) = \max_{\|v\|=1} v^\top H\, v. \tag{7}$$

This is the worst-case curvature in any direction; by (2), it sets the steepest climb of the training loss.

Hutchinson trace estimator $\mathrm{Tr}(H)$*:*

$$\mathrm{Tr}(H) \approx \frac{1}{K}\sum_{k=1}^{K} z_k^\top H\, z_k, \quad z_k \sim \text{Rademacher}(\pm 1)^d. \tag{8}$$

Each Hessian-vector product Hz_k is computed through two automatic-differentiation backward passes, with no dense Hessian formed. We use $K{=}50$ probes; preliminary runs at $K \in \{20, 100\}$ showed the estimate had stabilized by $K{=}50$.

Scale-normalized sharpness S_{norm}: Defined in (6); obtained by dividing the $\lambda_{\max}$ estimate by the squared ℓ_2 parameter norm.

B. Power Iteration for $\lambda_{\max}$

Algorithm 1 runs $T{=}100$ steps. The Rayleigh quotient converges to within 10^{-3} relative change in every run of our experiments. The Hessian itself is never stored; the only extra memory is a single gradient-shaped buffer beyond the model. Algorithm 2 reproduces the SAM update for reference, since S_{norm} can be read as the post-hoc measurement counterpart to SAM's training-time minimization.

C. Pearson Correlation as a Summary

We summarize the relationship between curvature and accuracy by the Pearson correlation r between optimizer-level mean values. With three optimizers per dataset the effective

Algorithm 1 Power iteration for $\lambda_{\max}$

Draw random v; set $v \leftarrow v/\|v\|$
for $t = 1$ to T **do**
  $u \leftarrow Hv$ ▷ two backward passes
  $v \leftarrow u/\|u\|$
end for
return $\hat{\lambda}_{\max} \leftarrow v^\top Hv$

Algorithm 2 SAM update [**?**] (for reference)

$g \leftarrow \nabla_\theta L(\theta)$
$\epsilon \leftarrow \rho\, g/\|g\|$
$g_{\text{SAM}} \leftarrow \nabla_\theta L(\theta+\epsilon)$
$\theta \leftarrow \theta - \eta\, g_{\text{SAM}}$

sample size is small, and no p-value will reach a conventional threshold. We therefore treat r as a directional indicator rather than a statistical test, in the same spirit as the cautious framing in [**?**]. What we read from the correlations is the direction and the change in sign across datasets; the absolute magnitudes are context-dependent.

V. Experimental Setup

A. Datasets

MNIST [**?**]—70,000 grayscale digit images at 28×28, ten classes. We use the standard 60,000/10,000 train–test split. Class boundaries are well separated and within-class variation is small, which gives us a low-complexity baseline.

CIFAR-10 [**?**]—60,000 RGB natural-image samples at 32×32, ten classes, with the standard 50,000/10,000 split. Inter-class similarity, background clutter, and color variation make CIFAR-10 substantially harder; as the experiments will show, this corresponds to a qualitatively different geometric regime.

We deliberately turn off data augmentation on both datasets. Augmentation introduces an implicit regularizer that would mask the effect of optimizer geometry on the converged curvature, and our experimental question is precisely about that effect. The trade-off is that absolute CIFAR-10 accuracy ends up below typical augmented benchmarks; we discuss this in Section VIII-C.

B. Architectures

MLP for MNIST. Three fully connected hidden layers with widths $[512, 256, 128]$ and ReLU activations, $\sim 567{,}000$ parameters in total. We omit batch normalization deliberately: BN's learnable scale parameters interact with the Hessian in the way [**?**] described, and including them on the simpler dataset would muddle the interpretation of $\lambda_{\max}$ and S_{norm}.

ResNet-18 for CIFAR-10. The standard 18-layer residual architecture from [**?**], modified for small inputs in the usual way: a 3×3 stem replaces the 7×7 stem, and the first max-pool is removed. ResNet-18 contains BN layers; we return to the implications in Section VII.

TABLE II
MNIST (MLP), MEAN ± STD OVER 5 SEEDS.

Optimizer	Acc. (%)	$\lambda_{\max}$	$\mathrm{Tr}(H)$	S_{norm}
SGD	93.09±0.06	6.93±0.72	44.86±3.21	0.148±0.015
Momentum-SGD	97.70±0.05	4.34±0.10	30.13±8.08	0.089±0.003
Adam	**97.88±0.23**	**2.84±1.56**	**5.32±3.09**	**0.057±0.031**

C. Optimizers and Training Protocol

We run three optimizers chosen to span a broad range of update behaviors:

- **SGD** (no momentum), $\eta = 0.01$.
- **Momentum-SGD**, $\eta = 0.01$, momentum $\mu = 0.9$.
- **Adam** [**?**], $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Cross-entropy loss, batch size of 128, and 20 training epochs were used for each setting. Neither of these confounds was used in our experimental settings, as they introduce factors that make it more difficult to attribute curvature difference solely to the optimizer. Each setting is run five times using different seeds, and we report mean ± standard deviations. All experiments were run in PyTorch using one GPU. For power iteration, we set $T = 100$, and $K = 50$ for the Hutchinson estimator.

VI. Results

A. MNIST: Curvature and Accuracy Move Together

The results from MNIST are clear-cut. Figs 1 through 4, combined with Table II, speak with one voice—the relative performance of the three optimizers does not depend on what column one sorts by. SGD has the most pronounced and least accurate trajectory; Momentum-SGD has moderate accuracy and pronouncedness; and Adam has flat trajectories and high accuracy. The fact that this relative order persists for $\lambda_{\max}$, $\mathrm{Tr}(H)$, and S_{norm} rules out the possibility that it is an artifact of one metric alone.

These numbers are certainly worth noting. For Adam, $\lambda_{\max} = 2.84$, compared to that for SGD, which is 6.93. This is a gap of $2.4\times$ in favor of Adam, and an accuracy of 4.8 points. An even larger gap appears when considering traces: 5.32 and 44.86 for Adam and SGD, respectively. This shows us that Adam's superiority in terms of flatness does not only hold for one particular direction but in general. Finally, S_{norm} for Adam is 0.057, while for SGD, it is 0.148.

The Pearson correlations on MNIST are

$$\begin{aligned} r(\text{Acc}, \lambda_{\max}) &= -0.943, \\ r(\text{Acc}, \mathrm{Tr}(H)) &= -0.804, \\ r(\text{Acc}, S_{\text{norm}}) &= -0.961. \end{aligned} \tag{9}$$

The improvement from -0.943 to -0.961 for S_{norm} over raw $\lambda_{\max}$ is small. It is, however, in the direction theory predicts: removing the parameter-scale factor brings the metric into closer alignment with accuracy.

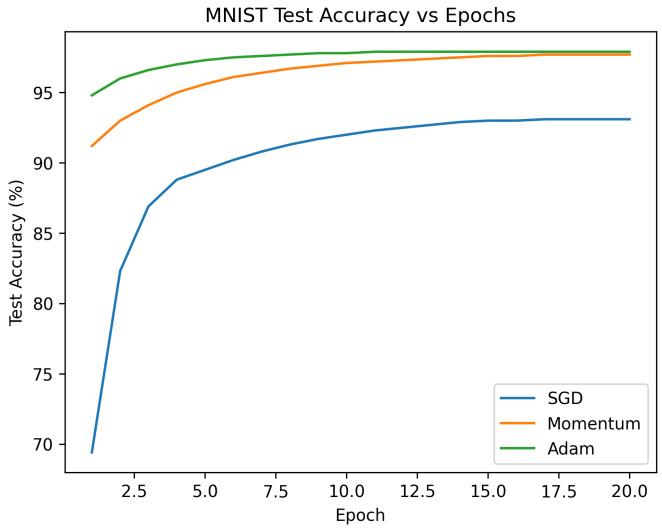


Fig. 1. Test-accuracy trajectories on MNIST over 20 epochs. Adam and Momentum-SGD plateau within the first five epochs; vanilla SGD converges noticeably more slowly and to a lower final accuracy.

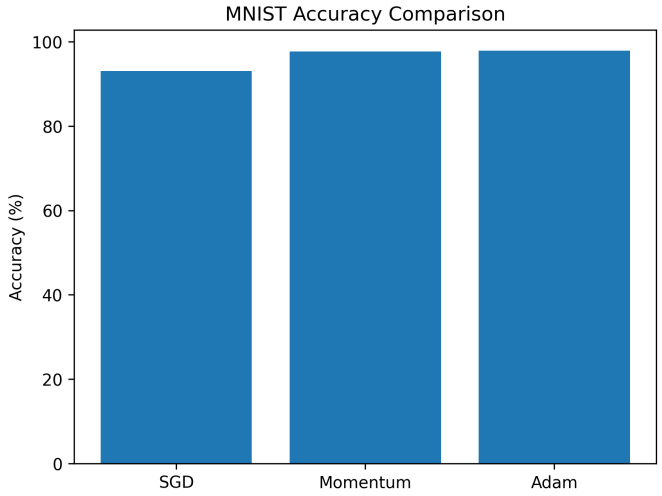


Fig. 2. MNIST accuracy summary by optimizer (mean ± std, five seeds). Error bars are tight enough that the three groups do not overlap.

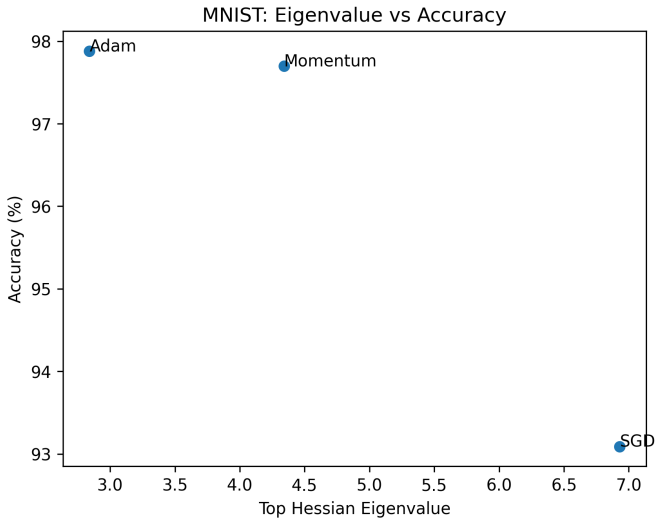


Fig. 3. MNIST: dominant eigenvalue against test accuracy. Each marker is one optimizer (mean values), with ±std bars in both directions. Pearson $r = -0.943$.

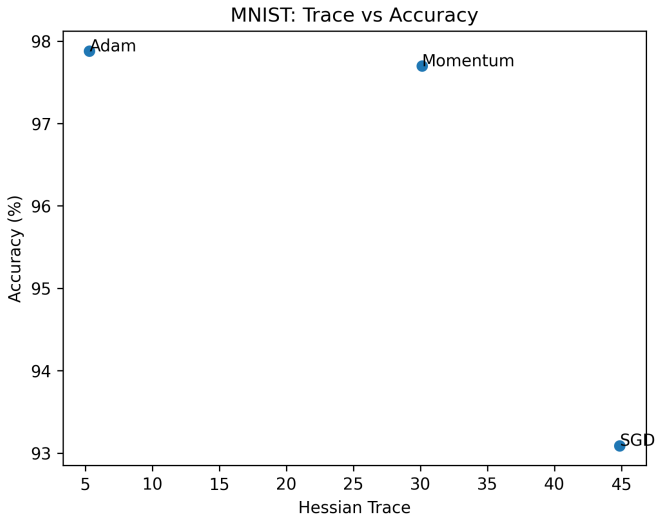


Fig. 4. MNIST: Hessian trace against test accuracy. The negative trend ($r = -0.804$) echoes the eigenvalue picture and suggests the broader curvature view is consistent with its dominant-direction summary.

B. CIFAR-10: The Coupling Vanishes

Table III resists the kind of summary that Table II invited. There is no single ordering across the columns. Momentum-SGD reaches 73.63%, almost ten points above SGD's 63.70%; yet its $\lambda_{\max}$ averages 125.88, which is $5.6\times$ SGD's 22.30. Adam achieves the highest accuracy (76.79%) at a curvature value that sits next to SGD's, not next to the flattest setting. Put plainly: the optimizer that generalizes second-best is also the sharpest, which is the opposite of what the flat-minima hypothesis would lead us to expect. Critically, the same inversion is visible in S_{norm} (Momentum at 0.174 versus Adam at 0.034 and SGD at 0.031), so this is not a parameter-scale illusion that S_{norm} can correct.

The Pearson correlations on CIFAR-10 are

$$\begin{aligned} r(\text{Acc}, \lambda_{\max}) &= +0.307, \\ r(\text{Acc}, \text{Tr}(H)) &= +0.073, \\ r(\text{Acc}, S_{\text{norm}}) &= +0.241. \end{aligned} \tag{10}$$

Compared to (9), every sign has flipped, and no magnitude exceeds 0.31. Scale normalization does not bring back the predictive signal; the breakdown on CIFAR-10 is structural.

C. Summary Across Datasets

The difference between them has been summarized in Table IV. In the case of CIFAR-10, the correlations were not only lower than in MNIST; they had the opposite signs! In that

TABLE III
CIFAR-10 (RESNET-18), MEAN ± STD OVER 5 SEEDS.

Optimizer	Acc. (%)	$\lambda_{\max}$	$\mathrm{Tr}(H)$	S_{norm}
SGD	63.70±0.55	22.30±11.21	94.45±31.42	0.031±0.016
Momentum	73.63±0.28	125.88±138.43	238.54±263.74	0.174±0.191
Adam	**76.79±0.56**	24.90±10.05	52.40±25.33	**0.034±0.014**

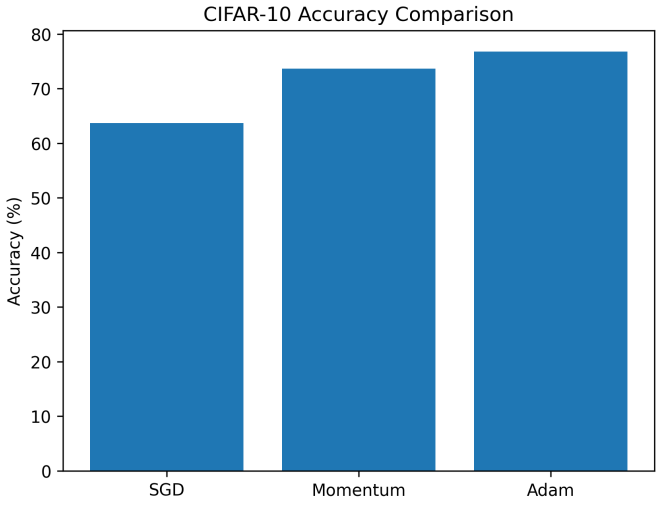


Fig. 5. CIFAR-10 test accuracy by optimizer. The accuracy gain from SGD to Adam is real, but it cannot be attributed to lower curvature.

sense, the differences are qualitative rather than quantitative. S_{norm} has the highest accuracy when curvature matters, but it is unable to help when there is no connection anymore.

VII. ANALYSIS

A. What Causes the Divergence?

Landscape geometry depends on task complexity: For MNIST, the three algorithms find themselves in distinct basins, which for the most part have been classified as narrower for SGD, wider for Momentum-SGD, and widest for Adam. This sequence can be geometrically explained by the uniformity of the landscape. The situation is quite different with CIFAR-10, where there are many similarly deep basins, each with its own curvature and not just the same flatness. And the basin that any particular algorithm will find itself in depends on its dynamics rather than on some preference for certain kinds of landscapes.

Where you land vs. how you got there: The variability numbers on CIFAR-10 are striking. Momentum-SGD's $\lambda_{\max}$ has a standard deviation of 138.43, which is *larger than its mean* of 125.88; meanwhile its accuracy standard deviation is just 0.28%. The implied coefficients of variation are ≈ 1.10 for curvature and ≈ 0.004 for accuracy—a $275\times$ mismatch. Reading this naturally: the five seeds converge to geometrically very different points and yet deliver almost identical test accuracy. Whatever advantage Momentum has over vanilla SGD on CIFAR-10, it is much more plausibly tied to the path through parameter space (in the spirit of [**?**]) than to where the path stops.

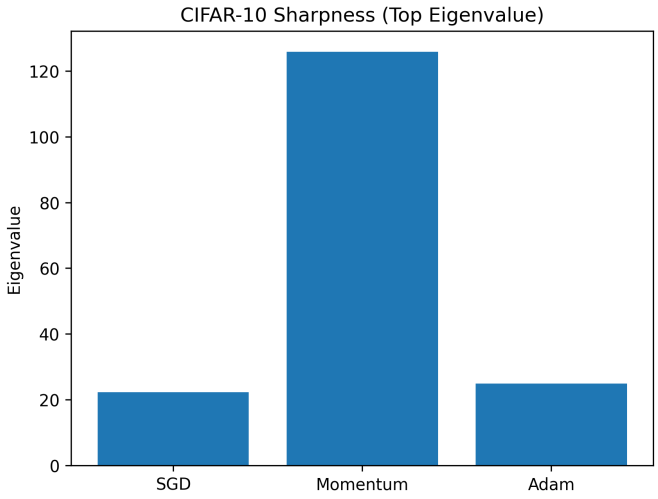


Fig. 6. Dominant eigenvalue per optimizer on CIFAR-10 (mean ± std). Momentum-SGD's bar towers over the other two, yet Momentum-SGD generalizes *better* than SGD.

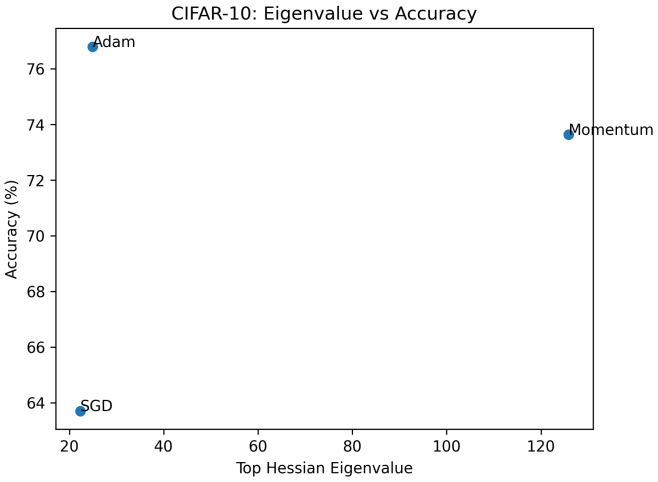


Fig. 7. CIFAR-10: $\lambda_{\max}$ against test accuracy. The slope is positive (r=+0.307), opposite to the MNIST trend; the relationship has changed sign.

Batch normalization tangles up curvature with scale: The ResNet-18 we use on CIFAR-10 contains BN. The learnable scale parameters in BN reproduce the same kind of homogeneity that makes raw $\lambda_{\max}$ ambiguous in the first place [**?**]. S_{norm} divides by the global $\|\theta\|^2$, which corrects only the uniform projection of that ambiguity. A layer-wise normalization would be more principled, and we discuss it as future work (Section IX).

B. What Adam's CIFAR-10 Advantage Tells Us

Adam outperforms SGD on CIFAR-10 by over thirteen percent (76.79% compared to 63.70%), yet its $\lambda_{\max}$, measured at 24.90±10.05, is no different from that of SGD (22.30±11.21). Curvature should explain this difference if it was the reason for higher accuracy. But Adam's $\lambda_{\max}$ is not lower than SGD's,

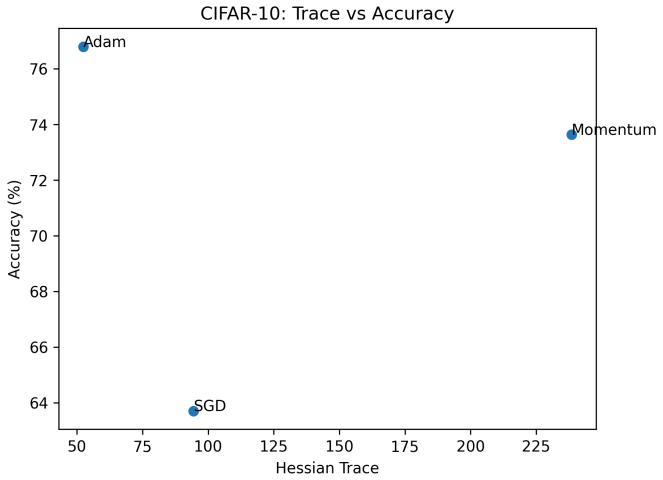


Fig. 8. CIFAR-10: Hessian trace against test accuracy (r=+0.073). A correlation this close to zero says that aggregate curvature carries essentially no predictive information about accuracy in this regime.

TABLE IV
PEARSON CORRELATION: CURVATURE VS. TEST ACCURACY.

Dataset (model)	$r(\text{Acc}, \lambda_{\max})$	$r(\text{Acc}, \text{Tr}(H))$	$r(\text{Acc}, S_{\text{norm}})$
MNIST (MLP)	-0.943	-0.804	$\mathbf{-0.961}$
CIFAR-10 (ResNet-18)	$+0.307$	$+0.073$	$+0.241$

so Adam does not reach a significantly flatter minimum than SGD. The main reason behind Adam's higher performance on CIFAR-10 appears to be its adaptive learning rate mechanism, which regulates the variance of gradients

C. The Practical Value (and Limits) of S_{norm}

On the MNIST dataset, going from the basic form of $\lambda_{\max}$, where $r = -0.943$, to S_{norm}, where $r = -0.961$, represents only a minor numerical improvement. Yet, the direction is consistent: without the scaling factor for the parameters, S_{norm} gets closer to the structure of the bound in (4). For the CIFAR-10 dataset, normalization results in r changing from +0.307 to +0.241. This also represents a modest numerical improvement; yet, the direction is wrong. The takeaway message is an uncomfortable one, but very straightforward: S_{norm} can help only when curvature has something to do with generalization.

VIII. Discussion

A. A Context-Dependent View of Flatness

The experiments conducted reveal a limit that had been assumed until now but never proven under controlled conditions before. Under conditions of lower task complexity, loss function curvature does indeed encode information pertaining to generalization ability, while beyond a certain point, heterogeneity increases to such an extent that curvature can no longer be regarded as a proxy. While the MNIST findings reinforce the original "flat minimum" thesis proposed by [**?**], [**?**], the CIFAR-10 findings support the scepticism expressed by [**?**], [**?**]. In essence, both perspectives are correct depending on the context.

B. Recommendations for Practitioners

The combined evidence supports four practical recommendations.

1) Report S_{norm} alongside raw $\lambda_{\max}$ when comparing curvature across optimizers or across model sizes. Raw $\lambda_{\max}$ can mislead if the parameter scales differ, and the cost of computing S_{norm} is zero on top.
2) For tasks of low to moderate complexity, where the regime is closer to MNIST than to CIFAR-10, treat curvature metrics as informative diagnostic tools. Flatness-aware methods such as SAM [**?**] are likely to help in these settings.
3) For harder tasks, do not assume flatness-aware methods will generalize better. Validate empirically on test accuracy. A drop in curvature is not, by itself, evidence of a benefit.
4) When an optimizer shows high curvature variance across seeds together with stable accuracy—as Momentum-SGD does on CIFAR-10 here—measure algorithmic stability [**?**] alongside curvature. Stability is the more informative quantity in such regimes.

C. Limitations

Several limitations bear stating plainly.

First, the empirical scope is two datasets and two architectures. Conclusions about larger models such as vision transformers or GPT-style networks, and about harder benchmarks such as CIFAR-100 or ImageNet, are not justified by the present data. Second, training runs only twenty epochs, with no learning-rate warmup, no cosine decay, and no weight decay—all standard in competitive setups. The absolute CIFAR-10 accuracy is well below state-of-the-art numbers as a result. The choice was deliberate (we want to isolate the optimizer's geometric signal), but it does mean the converged points are not as close to a real minimum as they would be in practice. Third, S_{norm} uses a global division by $\|\theta\|^2$, which corrects only the uniform component of the ReLU rescaling symmetry; a layer-wise variant would be more principled. Fourth, three-optimizer Pearson correlations have low statistical power, and our results should be read as directional evidence rather than statistical claims.

IX. Future Work

The most pressing extension is a layer-wise version of S_{norm}, which would account for the very different parameter magnitudes across layers in residual networks with BN. Beyond that:

- Test whether SAM implicitly minimizes a quantity close to S_{norm}, and whether that explains its empirical accuracy gains.
- Extend the study to vision transformers, where loss-landscape geometry is largely uncharted.
- Connect S_{norm} to the double-descent phenomenon [**?**]: does normalized curvature show a predictable pattern across the two descent regimes?

- Run the full study on CIFAR-100, TinyImageNet, and ImageNet under modern training schedules with augmentation.
- Derive tighter PAC-Bayes bounds using S_{norm} directly, following the structure of (4).

X. Conclusion

We set out to address two interlocking problems with curvature-based generalization measures: their sensitivity to parameter rescaling, and the absence of a controlled study mapping the conditions under which they predict generalization. Our scale-normalized metric $S_{\text{norm}}{=}\lambda_{\max}(H)/\|\theta\|^2$ tackles the first by construction (with the layer-wise refinement deferred), and a two-dataset experiment tackles the second empirically.

The results speak for themselves. For the MNIST benchmark, the raw and normalized curvature measures exhibit correlations with accuracy in the range -0.80 to -0.96, and the rankings of optimizers based on flatness perfectly coincide with their accuracies. On the other hand, all correlations on CIFAR-10 drop below 0.31 and the signs reverse. Normalizing the measures by scale improves the MNIST correlations a little (-0.943 to -0.961) but leaves the CIFAR-10 breakdown untouched. This shows that the breakdown is an intrinsic property of the landscape, not a flaw in the measurements. The momentum-SGD algorithm's decoupling of curvature and accuracy variances on CIFAR-10 suggests that trajectory dynamics are the key variable there.

In summary, practitioners should simply adopt S_{norm} as a replacement for $\lambda_{\max}$ when comparing different optimizers. But, as always, they must check first that the curvature-accuracy relationship holds on their target problem before attempting flatness-guided training.

Reproducibility Statement

All experiments use fixed seeds $\{0, 1, 2, 3, 4\}$ for both PyTorch and NumPy. Hyperparameters are listed in Section V. Code, training logs, and serialized checkpoints sufficient to reproduce every entry of Tables II and III will be released on publication.